

Paginación del carrito de compras e idempotencia de ventas

2026-10-04 · V6 · Continúa del documento 06 (Venta–Usuario e Historial de Caja). Este documento explica qué cambió desde aquel PDF hasta aquí: por qué un doble "Enter" en el modal de cobro creaba dos ventas, cómo se resolvió en las tres capas (base de datos, backend y frontend) y cómo se verificó contra la base de datos real.

Capa	Qué se toca	Archivo
Base de datos	Columna idempotency_key + índice UNIQUE parcial	V6__ventas_idempotencia.sql
Backend	Comprobación de la clave antes de cobrar y recuperación de la carrera	SaleImpl, SaleController, SaleRepository
Frontend	Bandera de vuelo + clave estable por intento de cobro	sale-facade.ts, cobro.html

Estado al cierre de este documento

Migración V6 **aplicada** en la base de datos real (Supabase) y registrada por Flyway. Backend: **273 pruebas** en verde (262 previas + 11 nuevas de idempotencia). Frontend: build de desarrollo en verde. La prueba de extremo a extremo contra la base real está documentada en la sección 9.

Sobre la paginación del carrito

Hay **dos** carritos y los dos acabaron paginados, pero en momentos distintos. El de **compras** (modal de nueva compra) se paginó en la V5 y está en el **documento 06**, sección 2. El de **venta** (el carrito del POS, en `cobro.html`) se pagina aquí, en la sección 2. Los dos se ven iguales al final, pero el del POS pasó cuatro meses con la decisión escrita de "no paginarlo", y esa decisión estaba equivocada: se explica abajo por qué.

1. El problema: un doble Enter cobraba dos veces

El modal de confirmar venta del POS es un `<form>`. El cajero recibe el efectivo, escribe el monto recibido y aprieta Enter. Eso envía el formulario y se registra la venta. El problema aparece cuando el Enter se aprieta dos veces.

La consecuencia no era un mensaje duplicado: era una venta duplicada. Concretamente, y esto es lo que hace grave el bug:

- **El stock se descuenta dos veces.** El detalle de venta se guarda dos veces, así que el inventario baja el doble de lo que salió del local.
- **Se emiten dos tickets.** El cliente (o el cajero) recibe un comprobante por cada pulsedón.
- **Los reportes mienten.** El corte de caja suma dos ventas por una sola compra, y el dinero de la caja aparece descuadrado contra el total real.

Es decir: el daño no era una molestia visual, sino una inconsistencia en los números que luego alimentan el inventario y los reportes. Por eso la solución tiene que vivir en el servidor y no en el botón.

1.1. Qué NO era el problema

No era un problema de validaciones del formulario, ni de que el botón "no se deshabilitara", ni de la doble ejecución del evento `ngSubmit`. El formulario hacía exactamente lo que se le pedía: enviar. El defecto estaba en que **enviar dos veces dos veces creaba dos ventas**, y eso es una decisión del servidor, no de la pantalla.

1.2. La evidencia medida

Antes de tocar código se reprodujo el problema contra la base de datos real con dos peticiones **simultáneas** y la misma clave. El resultado sin la protección, y la diferencia con la protección aplicada, están en la sección 9.

2. El carrito de venta (POS): por qué sí se pagina ahora

El encargo original era "paginar el carrito", y había dos carritos distintos con el mismo nombre. El de **compras** ya estaba hecho desde la V5 (documento 06). Este es el del **POS**: la tabla de productos que el cajero arma antes de cobrar, en `cobro.html`.

2.1. El problema: la tabla no tenía tope

El carrito del POS se dibuja con `*ngFor` sobre **todos** los productos del carrito, sin ningún recorte. Con treinta productos la tabla crecía treinta renglones hacia abajo y empujaba el botón verde **Cobrar** fuera de la pantalla: el cajero tenía que scrollear para llegar al botón con el que cobra. En un POS eso no es una molestia estética, es tiempo por venta.

2.2. La decisión anterior, y por qué estaba equivocada

Una decisión escrita que este encargo revierte

El 2026-09-12 quedó escrito en el `AGENTS.md` del frontend: *"El carrito del POS no se pagina: es un carrito vivo, no una tabla de registros"*. La razón tenía sentido, pero confundía dos cosas distintas. El carrito **sigue siendo vivo** y se sigue pareciendo como tal: el renglón conserva sus botones de `+/-X` y la cantidad se sigue tocando en el sitio. Lo que estaba mal era la **altura**: "vivo" no significa "sin tope". Paginarlo no lo convierte en una tabla de registros; le pone un límite a la altura.

2.3. Lo que se implementó

```
sale-facade.ts
cartPage      = 0;
cartPageSize = 8;    // 8 y no 5: aquí el renglón es UNA línea
```

```

get cartTotalItems() { return this.cobro.cart$.value.length; }
get cartTotalPages() { return Math.max(1, ceil(total / 8)); }
get cartDesde()      { return cartPaginaValida * 8 + 1; }
get cartHasta()      { return min(desde + 7, total); }

cobro.html
*ngFor="let item of (sale.cobroItems$ | async)
    | paginate: sale.cartPage : sale.cartPageSize"
*ngIf="((sale.cobroItems$ | async) ?? []).length > sale.cartPageSize"

```

8 renglones y no 5, a propósito

En el carrito de compras el renglón tiene seis columnas y cuatro input, así que es alto y caben cinco. En el carrito del POS el renglón es **una sola línea**, así que caben ocho. Copiar el 5 del otro carrito por costumbre habría dejado el carrito del POS más bajo de lo necesario.

2.4. Reutilización: el MISMO PaginatePipe

La lógica de paginación **no se duplicó**. El carrito usa el PaginatePipe de core/pipes/paginate, el mismo que los CRUDs, y el componente solo lo importa. Además ese pipe ya resolvía por su cuenta el caso de una página fuera de rango (si se borra el último renglón de la página 2, se recorta a la última válida).

Un ajuste de tipo que hizo falta (y por qué no era cosmético)

Al escribir `(sale.cobroItems$ | async) | paginate: ...` el build falló con TS2345: el pipe async devuelve null antes del primer valor y la firma del pipe era `T[]`. La firma se cambió a `T[] | null | undefined`. No es una concesión al compilador: el cuerpo del pipe **ya** comprobaba ese caso en su primera línea (`if (!items?.length)`). El tipo solo describía menos de lo que el código hacía, y esa diferencia obligaba a escribir `| async ?? []` en cada template.

2.5. Los tres detalles que hacen que se comporte bien

Pagar el doble de una línea de código son los tres casos en los que la paginación "parece no hacer nada". Los tres están resueltos en la facade:

Detalle	Qué pasa si falta
irAPaginaDelProducto(id) al agregar, incluido el escáner	Escanear el producto 12 con la vista en la página 1 lo agrega invisible : el carrito "no cambia" a ojos del cajero, que es justo cuando más engaña
ajustarPaginaAlQuitar() al quitar o bajar	Borrar el último renglón de la página 2 deja la vista en una página que ya no existe: carrito vacío
cartPage = 0 tras <code>cobro.clear()</code>	Tras cobrar un carrito que estaba en la página 3, la venta siguiente aparece "en la página 3" de un carrito de 1 producto

Por qué se calcula por índice real y no "saltar a la última página"

Agregar un producto **que ya estaba** en el carrito no crea renglón nuevo: solo le sube la cantidad. Si ese renglón está en la página 1 y el carrito va en la página 3, saltar a la última página dejaría al cajero mirando la página equivocada. `irAPaginaDelProducto()` calcula `Math.floor(indice / cartPageSize)` sobre el **índice real** del producto, que acierta en los dos casos.

2.6. El pie no puede mentir

El paginate se recorta solo cuando la página queda fuera de rango, pero los números del pie ("Mostrando 9–16 de 24") se calculan aparte. Si una ruta futura quitara productos sin llamar al ajuste, el pie podría llegar a mostrar "Mostrando 9–3 de 3": un rango al revés.

Por eso los getters del rango usan `cartPaginaValida` (la página recortada al rango válido) y no `cartPage` directo. Si algún día hay un desajuste, el texto sigue siendo correcto. El botón "Siguiente" sigue moviendo `cartPage`, que es el estado real, y por eso el contador de página sí lo muestra tal cual.

El pie solo aparece con más de 8 productos

El `*ngIf` compara el total contra `cartPageSize`. En una venta de dos o tres productos un control de paginación es ruido, y lo que importa ahí es el botón verde Cobrar. También explica por qué en una venta corta no se ven las flechas: no es que falten. Y se añadió una fila de "carrito vacío", que antes no existía: con la tabla recortada, un carrito sin productos es un hueco más notorio.

2.7. Lo que NO se cambió

- El pie **no** usa el componente `<app-pagination>` compartido, aunque se podría. Ahí los controles son la navegación principal de una tabla; en el POS son secundarios frente al botón verde Cobrar, y el estilo del componente compartido (barra de tabla con borde superior) compite visualmente con él. Se replica su estructura con las clases `.cart-paginacion`, que siguen la misma paleta que el pie del carrito de compras.
- Las flechas usan `izquierda.png` y `derecha.png` con `filter: brightness(0) invert(1)`. Es seguro aquí porque esos dos iconos son de un solo color: el mismo filtro en `anadir.png`, que tiene cruz verde y contorno negro, los fundiría en un bloque blanco macizo.

3. Qué es la idempotencia y por qué es la respuesta correcta

Idempotencia es la propiedad de una operación según la cual aplicarla N veces tiene el mismo efecto que aplicarla una. Es exactamente lo que necesita un cobro: si la misma intención de cobro llega dos veces, el efecto debe ser **una sola venta**.

No es un invento nuestro: es el patrón estándar de las pasarelas de pago (Stripe lo llama `idempotency key`), y por eso el nombre de la columna y del campo es ese.

3.1. La clave la genera el CLIENTE, no el servidor

Esta es la decisión de diseño de la que depende todo lo demás

Si el backend generara la clave en cada petición, cada una sería distinta y la protección **no serviría de nada**: el servidor reconocería cada petición como una venta nueva. La clave tiene que identificar la **intención de cobrar**, y esa intención vive en el navegador del cajero. Por eso la columna es **nullable** y no tiene `DEFAULT`: las ventas anteriores a V6 y las que se creen por otros caminos simplemente no la llevan.

3.2. Cuándo se genera y cuándo se reutiliza

Momento	Qué pasa con la clave
Se abre el modal de cobro	Se genera una clave nueva . Cada apertura es una venta nueva y merece su propia clave.
Se aprieta Enter	Se reutiliza la misma clave. Da igual cuántas veces se apriete.
Se paga con tarjeta	Se reutiliza la misma clave: es el mismo cobro, solo cambia el método.
Venta confirmada	La clave muere con ese cobro. La siguiente venta genera otra.

El detalle que arregla el bug

La clave se genera al **abrir** el modal, no al **confirmar** el pago. Si se generara en el clic de confirmar, cada Enter tendría su propia clave y cada una sería una venta nueva: exactamente el problema que se quería evitar. El código lo deja escrito en `openPaymentModal()` para que nadie lo "simplifique" moviéndolo.

4. Capa 1 — Frontend: dos barreras en la interfaz

En el frontend hay dos barreras, y conviene nombrarlas por separado porque **no hacen lo mismo**.

4.1. Barrera 1: la bandera de vuelo

```
// sale-facade.ts
isProcessing = false;

confirmPayment(){
  if(this.isProcessing){ return; }      // <-- barrera 1
  ...
  this.isProcessing = true;
  this.saleService.processSale(request).subscribe({
    next: (r) => { this.isProcessing = false; ... },
    error: (e) => { this.isProcessing = false; ... }    // se libera
                                                         // también en error
  });
}
```

La bandera se libera **tanto en el éxito como en el error**. Eso es un detalle pequeño con consecuencia real: si solo se liberara en el éxito, un cobro rechazado dejaría el modal bloqueado y el cajero no podría ni reintentar ni cerrar.

```
// cobro.html – el botón AVISA
<button type="submit" [disabled]="sale.isProcessing">
  {{ sale.isProcessing ? 'Cobrando...' : 'Confirmar pago' }}
</button>
```

4.2. Barrera 2: la clave que viaja al backend

```
// sale-facade.ts
private claveCobro = '';

openPaymentModal(){      // se genera AQUÍ, no al confirmar
  ...
  this.iniciarIntentoDeCobro();
  this.showPaymentModal = true;
}

confirmPayment(){
  const request: SaleRequest = {
    paymentMethod: this.selectedPaymentMethod,
    cashReceived: this.cashReceived,
    idempotencyKey: this.claveCobro,      // <-- barrera 2
    items: ...
  };
}
```

La generación del identificador usa `crypto.randomUUID()` cuando existe, con un respaldo basado en `Math.random()` para entornos sin esa API. El respaldo **no es criptográficamente seguro** y el código lo dice: aquí la clave sirve para evitar un cobro doble, no para proteger un secreto. Si algún día se usara para algo sensible, esa nota es la advertencia de que hay que cambiarlo.

4.3. El flujo de tarjeta

El modal de tarjeta tiene su propia bandera, `cardProcessing`, y la misma clave. Se comparte a propósito: elegir tarjeta y luego efectivo sigue siendo **el mismo cobro**, así que si el cajero cerrara el modal de tarjeta y pagara en efectivo, el backend sigue reconociéndolo como el mismo intento y no cobra dos veces.

Por qué la barrera 1 no alcanza (y por qué está la 2)

El botón deshabilitado evita el segundo **click**, pero no el caso real: las dos peticiones ya pueden estar viajando por

la red al mismo tiempo. Además, si el usuario recarga la pestaña, cambia de ventana o reintenta porque la respuesta tardó, la venta se volvería a crear. La barrera 1 protege la **interfaz**; la barrera 2 protege la **operación**, y por eso la que resuelve el problema es la segunda.

5. La capa que de verdad garantiza: la base de datos

Ni el frontend ni una comprobación previa en JavaScript garantizan que no haya doble venta. La garantía real es una restricción de la base de datos.

```
-- V6__ventas_idempotencia.sql
ALTER TABLE public.sales
  ADD COLUMN IF NOT EXISTS idempotency_key varchar(64);

CREATE UNIQUE INDEX IF NOT EXISTS idx_sales_idempotency
  ON public.sales (idempotency_key)
  WHERE idempotency_key IS NOT NULL;
```

5.1. Por qué UNIQUE y no un índice normal

Un índice común solo **acelera** la búsqueda: dos ventas con la misma clave seguirían existiendo. Un UNIQUE **rechaza** el duplicado. Son cosas distintas y confundirlas deja el bug abierto, porque la aplicación se vería correcta (la búsqueda funciona) mientras el stock se sigue descontando dos veces.

5.2. Por qué el índice es PARCIAL

En PostgreSQL un UNIQUE normal ya permite varios NULL, porque NULL != NULL. El filtro WHERE idempotency_key IS NOT NULL no es necesario para la corrección: es una optimización.

Se deja igual porque la tabla de ventas es la más grande del sistema y casi todas sus filas tienen la columna en NULL (ventas viejas, ventas de otros flujos). Un índice parcial sobre las pocas filas que sí la tienen es mucho más pequeño y rápido de mantener, y es exactamente lo que se va a consultar.

Por qué no se acortó el nombre de la columna

El nombre largo idempotency_key es el que usa la documentación de Stripe y el que aparece en los logs. Una abreviatura como idem_key llenaría menos, pero obligaría a traducirlo cada vez que se lee una integración externa. En tablas de 14 columnas el ahorro de dos caracteres no paga ese costo.

6. Capa 2 — Backend: comprobar antes de cobrar

El servicio recibe la clave por el DTO, la busca, y solo cobra si no existe.

6.1. Las clases y cómo se relacionan

Clase	Qué aporta	Relación
SaleRequest (DTO de entrada)	El campo idempotencyKey que recibe la clave	Lo llena el frontend
SaleEntity	El campo idempotencyKey que la persiste	Se copia desde el DTO
SaleRepository	findByIdempotencyKey()	Interroga a la tabla
SaleImpl	Comprueba la clave, normaliza y decide	Orquesta el caso normal
SaleService	El método de relectura para la carrera	Contrato, para el controlador
SaleController	Recupera la carrera perdida	Última línea de defensa

6.2. El orden de las operaciones (esto es lo importante)

La comprobación va **primero, antes de validar el carrito y antes de crear la venta**. Si se hiciera más tarde, la venta loser ya habría descontado stock antes de detectar que era repetida: la protección serviría de adorno.

```
// SaleImpl.processSale(request)
if(ventaPrevia != null){
    log.info("Venta {} ya procesada con la clave de idempotencia. "
        + "Se devuelve sin cobrar de nuevo.", ventaPrevia.getId());
    return mapper.toResponse(ventaPrevia);    // <-- atajo temprano
}
validateRequest(request);
CashRegisterEntity cash = getActiveCashRegister();
SaleEntity sale = createBaseSale(request, cash);    // aquí va la clave
```

6.3. Normalizar la clave antes de usarla

```
private String normalizarClave(String clave){
    if(clave == null || clave.isBlank()){ return null; }
    String limpia = clave.trim();
    return limpia.length() > 64 ? limpia.substring(0, 64) : limpia;
}
```

- **Recortar** los espacios: un salto de más convertiría la misma intención de cobro en dos claves distintas, que es justo lo que la idempotencia debe evitar.
- **Tratar el blanco como ausente**: buscar por cadena vacía haría que todas las ventas sin clave colisionaran entre sí.
- **Acotar a 64**, que es el ancho de la columna: un cliente que mande algo enorme no debe romper el INSERT.

6.4. Un nombre DERIVADO, y el error que casi se repite

Por qué el repositorio se llama findByIdempotencyKey

Spring Data deriva el nombre de la consulta de la propiedad. `findBy` + `IdempotencyKey` resuelve a la propiedad `idempotencyKey` de la entidad, que existe. El error similar ya se cometió una vez en este proyecto, en `CashBoxRepository`: `existsByCashRegistersId` no funcionaba porque del lado de la relación la propiedad se llama `cashRegister`, no `cashRegisters`. La regla que dejó esa nota sigue vigente: si el nombre derivado no compila, el problema suele estar en el nombre de la propiedad, no en la consulta.

7. El caso que la comprobación previa no cubre

Todo lo anterior resuelve el doble Enter **secuencial**: la primera petición terminó y creó la venta, la segunda la encuentra y la devuelve. Pero hay un caso peor: las dos peticiones **al mismo tiempo**.

7.1. Por qué la comprobación previa no basta

```
petición A:  buscar clave -> no existe  ...  insertar
petición B:  buscar clave -> no existe  ...  insertar
```

Las dos buscan antes de que ninguna escriba. Es una carrera clásica: cada una ve un mundo donde la otra todavía no existe.

7.2. El UNIQUE resuelve la carrera

Con el índice UNIQUE, PostgreSQL deja pasar a una de las dos y rechaza a la otra. En ese momento el problema cambia: ya no hay venta duplicada (la perdedora revierte toda su transacción, stock incluido), pero el cajero vería un error 500 aunque su venta **sí** estuviera cobrada y registrada. Eso es peor que un bug: parece un fallo y además cobró.

7.3. El detalle no obvio de PostgreSQL (y por qué funciona)

Para recuperar la venta ganadora hace falta saber algo: cuando el INSERT perdedor choca con el índice, **no falla de inmediato**. PostgreSQL lo bloquea dentro del índice hasta que la transacción ganadora resuelve. El error de clave duplicada **solo se emite cuando la ganadora ya hizo COMMIT**.

Consecuencia práctica

Cuando el backend recibe el error de duplicado y relee la venta por esa clave, la fila **ya está confirmada y es visible**. Por eso esta recuperación no necesita reintentos ni esperas: no hay carrera al releer, porque el perdedor esperó a que el ganador terminara. Si alguien cambia a una base de datos sin este comportamiento, este código necesitará un ciclo de reintentos y esta nota habrá que rehacerla.

7.4. Dónde se recupera, y por qué en transacción nueva

```
// SaleController.processSale(request)
try {
    return ResponseEntity.ok(service.processSale(request));
}
catch(DataIntegrityViolationException e){
    return service.findByIdempotencyKey(request.getIdempotencyKey())
        .map(ResponseEntity::ok)
        .orElseThrow(() -> e); // <-- si NO es un cobro duplicado,
                                //      el error NO se esconde
}
```

La relectura se declara `@Transactional(readOnly = true)` a propósito, y ese es un detalle importante: se ejecuta **después** de que la transacción de la venta perdedora revirtió. Abrir una transacción nueva es lo que permite que la lectura sea válida; leer dentro de la transacción ya marcada como *rollback-only* lanzaría `UnexpectedRollbackException` en lugar de la venta.

Por qué el `orElseThrow` no es un detalle menor

Un error de integridad no siempre es un cobro duplicado: también puede ser una restricción de stock o un dato inválido. En esos casos no hay ninguna venta que devolver, y tragarse el error dejaría al cajero creyendo que cobró cuando no fue así. Si la relectura no encuentra nada, el error original se relanza tal cual.

Escenario	Qué hace el backend	Qué ve el cajero
Una sola pulsación	No encuentra la clave, cobra normal	Su venta y su ticket

Escenario	Qué hace el backend	Qué ve el cajero
Doble Enter secuencial	La 2ª encuentra la venta de la 1ª y la devuelve	Su venta, sin error
Dos peticiones simultáneas	El UNIQUE rechaza a una; la otra se relee y se devuelve	Su venta, sin error
Restricción que no es duplicado	No hay venta que devolver: relanza el error	El error real, no un falso éxito

8. Verificación con código (automatizada)

La protección se fijó con pruebas en dos niveles. Todas en verde: **273 pruebas** en el backend (262 previas + 11 nuevas).

8.1. SaleImplIdempotencyTest — la lógica del servicio

- **devuelve la venta existente y no crea otra.** El assert que de verdad importa es `verify(saleRepository, never()).save(...)`: si se guardara algo, habría venta doble.
- **tampoco toca el stock ni abre la caja.** Se comprueba que `productRepository.findById` no se llega a llamar, lo que prueba que el atajo está antes de validar el carrito.
- **crea la venta y guarda la clave** en el camino normal.
- **sin clave** (un cliente viejo) la venta se crea normal, y además no se consulta el repositorio.
- **normalización:** recorta espacios, trata el blanco como ausente y acota a 64 caracteres.

8.2. SaleControllerIdempotencyTest — la carrera

- **si el UNIQUE rechaza el INSERT, devuelve la venta ya registrada** con 200 en vez de un 500.
- **si no hay venta con esa clave, el error no se esconde.** Esta prueba blinda el comportamiento contrario: tapar un error de integridad que no era duplicado sería peor que el bug original.
- **un cobro normal no consulta la clave:** el camino feliz no debe pagar el costo de la búsqueda extra.
- **la venta original se procesa una sola vez:** en la recuperación solo se lee, nunca se vuelve a cobrar.

Una lección sobre las pruebas

Dos pruebas fallaron al escribirlas, y el motivo fue que el código era **mejor de lo que la prueba asumía**: con la clave ausente, el servicio no consulta el repositorio en absoluto (corta antes). La prueba afirmaba un stub que nunca se usaba, y Mockito la marcó como stubbing innecesario. La corrección fue **aceptar el comportamiento real** y cambiar el assert por `never()`, no relajar la prueba para hacela pasar.

9. Verificación contra la base de datos real

La migración se aplicó en Supabase el 2026-10-04 y **Flyway la registró sola** al arrancar (informando que ya existía, que es exactamente el comportamiento correcto de `IF NOT EXISTS`). Con la columna presente, la aplicación mapea el campo nuevo sin problemas.

9.1. Que el UNIQUE existe y funciona

Dos INSERT con la misma clave dentro de una transacción que se revierte:

```
BEGIN;
INSERT ... idempotency_key='PRUEBA-V6-DUPLICADO' ...;
INSERT ... idempotency_key='PRUEBA-V6-DUPLICADO' ...;
ROLLBACK;

ERROR:  duplicate key value violates unique constraint
        "idx_sales_idempotency"
DETALLE:  Key (idempotency_key)=(PRUEBA-V6-DUPLICADO) already exists.
```

9.2. El doble cobro, con números reales

Dos peticiones **en paralelo** (dos jobs al mismo tiempo, no una detrás de otra) con la misma clave, comprando 2 unidades de cada vez. Stock inicial del producto: **17**.

Qué se comprobó	Resultado
Respuesta de la 1ª petición	saleId 8 , total 42.00
Respuesta de la 2ª petición	saleId 8 , total 42.00 (la misma)
Ventas creadas con esa clave	1 (no 2)
Stock del producto	17 → 15 (bajó 2, no 4)
Renglones en el detalle	1 de cantidad 2

9.3. Y el caso secuencial

El caso real de un doble Enter: la segunda petición llega después de que la primera terminó. Con otra clave y una unidad, la 1ª llamada devolvió saleId=10 y la 2ª devolvió saleId=10. Una sola venta; el stock bajó de 15 a 14, una unidad.

9.4. Limpieza de los datos de prueba

Las dos ventas de prueba se **anularon por la API** (no se borraron a mano de la tabla), lo que devolvió el stock a su valor original, y la caja que se abrió para la prueba se cerró. Estado final verificado: producto con stock **17** y **cero** cajas abiertas. Las filas de venta se conservan como anuladas, que es lo que el propio sistema hace con una venta normal y por lo que el historial no miente.

Por qué se anulan y no se borran

El módulo de ventas es append-only por decisión de diseño (ver el documento 02): una venta no se borra, se anula. Borrar filas de prueba a mano dejaría huecos en los números de ticket y en los reportes. Anular por la API mantiene la auditoría coherente.

10. Decisiones que quedaron fuera (y por qué)

Se descartó	Por qué
Generar la clave en el backend	Cada petición tendría una clave distinta: no habría forma de reconocer el duplicado (sección 3.1).
Quitar el Enter del formulario	Escribiendo el monto con Enter es la forma natural de cobrar. Se corrigió la consecuencia, no la costumbre.
Bloqueo pesimista con SELECT ... FOR UPDATE	No protege nada: sobre una fila que todavía no existe no hay nada que bloquear. El UNIQUE resuelve la carrera sin serializar a todos los cajeros.
Tabla aparte de claves reservadas	Una reserva previa en tabla propia duplica el estado de la venta y deja transactions colgando si el proceso muere entre la reserva y el cobro.
Reintentos en la recuperación	En PostgreSQL no hacen falta: el error se emite tras el COMMIT de la ganadora (sección 7.3).

10.1. Lo que sigue pendiente

- **La migración V6 está aplicada en Supabase**, pero cualquier base de datos *nueva* la toma Flyway automáticamente al arrancar. No hay paso manual.
- **El POS sigue sin un reintento automático**: si el backend no responde, el cajero tiene que volver a apretar. Eso está bien, porque el segundo intento lleva la **misma clave** y no cobra dos veces: es justamente el escenario para el que se construyó esto.
- **Las alertas nativas (alert()) siguen siendo la forma de aviso** en el POS. Cambiarlo a MatSnackBar es una tanda aparte de consistencia visual; no se mezcló aquí porque no tiene relación con el doble cobro.

11. Índice de archivos tocados

Archivo	Qué se hizo
db/migration/V6__ventas_idempotencia.sql	Nuevo: columna + índice UNIQUE parcial
model/entity/SaleEntity.java	Nuevo campo idempotencyKey
model/dto/Sale/SaleRequest.java	Nuevo campo idempotencyKey (entrada)
repository/SaleRepository.java	Nuevo findByIdempotencyKey
service/SaleService.java	Nuevo findByIdempotencyKey (relectura)
service/impl/SaleImpl.java	Comprobación previa, normalización, atajo temprano, persistencia de la clave
controller/SaleController.java	Recuperación de la carrera simultánea
test/.../SaleImplIdempotencyTest.java	Nuevo: 7 pruebas del servicio
test/.../SaleControllerIdempotencyTest.java	Nuevo: 4 pruebas de la carrera
features/charge/facade/sale-facade.ts	Clave por intento, banderas de vuelo, envío de la clave
features/charge/cobro.html	Botón deshabilitado y etiqueta de "Cobrando..."

Archivo	Qué se hizo
core/interfaces/sale/sale.ts	Campo opcional idempotencyKey

Con esta V6 el cobro del POS tiene, por fin, la propiedad que necesitaba: pulsar Enter dos veces cobra una vez. Y la paginación del carrito deja de empujar el botón de guardar fuera de la pantalla.